



WORDLE CLONE - WORD GUESS BATTLE

Presented by:

Archana J - 192324294

Course Code: CSA0920RC

Faculty: DR. RamKumar M

Date: 01/08/2026



Game Concept Overview



- **What is the game?**

- Offline Wordle game developed using Java Swing.
- Player guesses a hidden five-letter word within six attempts.
- Each guess receives color-coded feedback.

- **Core Game Loop**

Start Game → Select Random Word → Enter Guess → Validate Guess → Show Feedback → Win / Next Guess → Game Over

- **Win Condition**

✓ Guess the correct five-letter word within 6 attempts.

- **Lose Condition**

✗ Fail to guess the word after 6 attempts.



Reference Game Study



- **Original Game – Wordle**

- Popular word puzzle game where players guess a five-letter word in six attempts.
- Gained popularity due to its simple rules, color-coded feedback, and engaging daily challenges.

- **Mechanics Reproduced**

- Random five-letter word selection.
- Six guessing attempts.
- Green, Yellow, and Gray tile feedback.
- Dictionary-based word validation.

- **Simplifications**

- Offline Java Swing desktop application.
- Local word dictionary instead of online daily updates.
- Basic score tracking using JDBC & MySQL (no online leaderboard or multiplayer).

- **Why These Changes?**

The project retains Wordle's core gameplay while reducing complexity, allowing the focus to be on implementing OOP, Collections, Swing GUI, Exception Handling, Multithreading, and JDBC effectively.



OBJECTIVES / LEARNING OUTCOMES TARGETED



- **Objectives**

- Develop an interactive Wordle Clone using Java Swing.
- Implement efficient word storage and lookup using Java Collections.
- Validate player guesses with color-coded feedback.
- Store player scores and winning streaks using JDBC & MySQL.
- Ensure smooth gameplay through proper Exception Handling and Multithreading.

- **Learning Outcomes**

- Apply Object-Oriented Programming (OOP) to design modular game components.
- Use Collections (ArrayList/HashSet) for dictionary management.
- Design an interactive Swing GUI with event handling.
- Implement JDBC for database persistence.
- Handle runtime errors using try-catch blocks.



Engineer to Excel



Proposed Module Breakdown

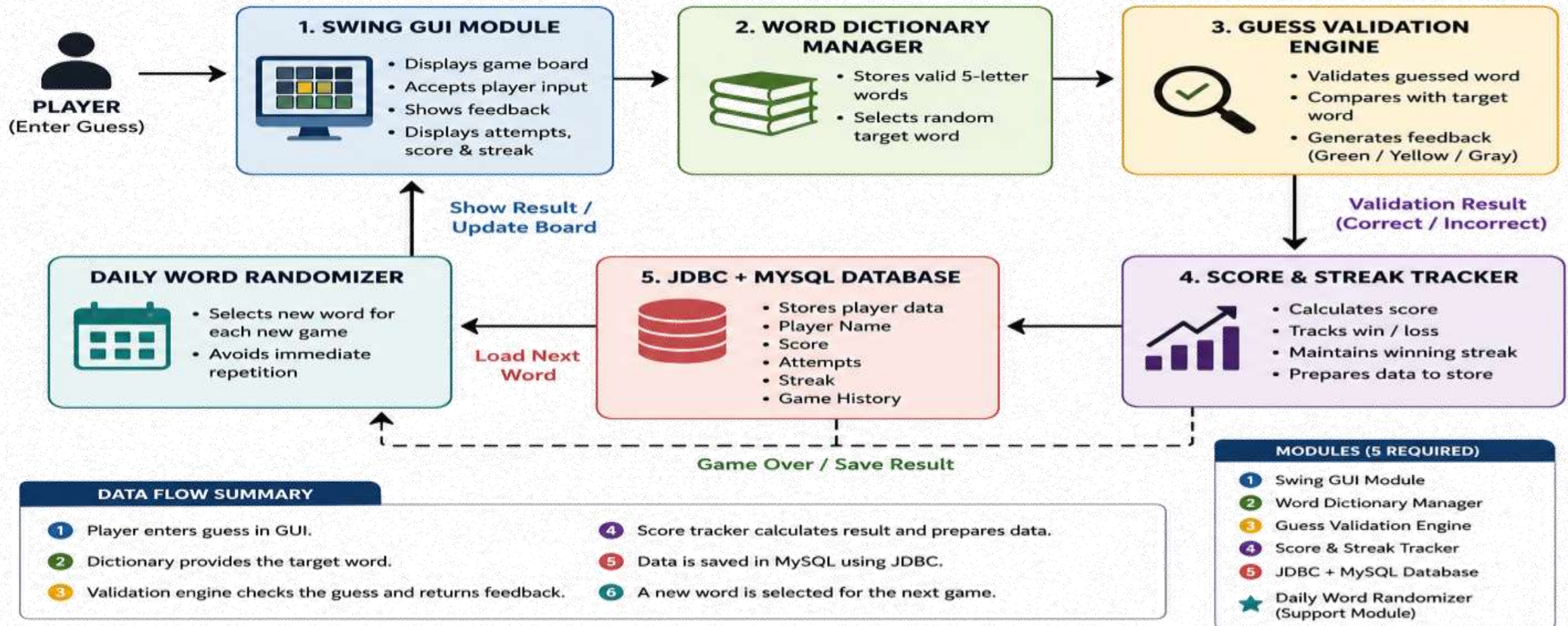
Module	Function	Core Java Feature / Data Structure
Word Dictionary Manager	Stores valid five-letter words and randomly selects the target word for each game.	ArrayList / HashSet, Random
Guess Validation Engine	Compares each guessed word with the target word and generates color-coded feedback.	String, Character Arrays, OOP
Swing GUI	Displays the game board, accepts user input, and updates colored tiles after each guess.	Java Swing, Event Handling (ActionListener)
Score & Streak Tracker	Records player name, score, attempts, and winning streak in the database.	JDBC, MySQL (Connection , PreparedStatement)
Daily Word Randomizer	Ensures a new target word is selected for every new game without immediate repetition.	Random, Collections (HashSet)

Java Concepts Demonstrated

OOP • Collections • Swing GUI • JDBC • Exception Handling • Multithreading

System Design / Architecture

WORDLE CLONE – WORD GUESS BATTLE





Engineer to Excel
Technology



Tech Stack & Why

	Why It Is Used in This Project
Java	Implements the core game logic and object-oriented design.
Java Swing/AWT	Builds the game interface, displays the word grid, keyboard, and color-coded feedback.
Collections Framework (ArrayList, HashSet)	Stores valid five-letter words and supports fast word lookup and random selection.
Object-Oriented Programming (OOP)	Organizes the game into reusable classes such as Game, Player, Dictionary, and Score Manager.
Exception Handling	Handles invalid inputs, missing dictionary files, and database connection errors without crashing the game.
Multithreading	Runs the game timer and GUI updates smoothly without freezing the interface.
JDBC + MySQL	Stores player names, scores, attempts, winning streaks, and game history after each game.

Key Technologies: Java • Swing/AWT • OOP • Collections • Exception Handling • Multithreading • JDBC • MySQL



Engineer to Excel



PROPOSED DATABASE SCHEMA

Table: `player_scores`

Column Name	Data Type	Description
<code>player_id</code>	INT (PK, AUTO_INCREMENT)	Unique player ID
<code>player_name</code>	VARCHAR(50)	Name of the player
<code>score</code>	INT	Score achieved
<code>attempts</code>	INT	Number of guesses used
<code>result</code>	VARCHAR(10)	Win / Loss
<code>streak</code>	INT	Current winning streak
<code>played_date</code>	DATETIME	Date and time of the game

When is Data Stored?

- **On Game Over:** Saves the player's score, attempts, result, and streak.

Database Flow





MULTITHREADING PLAN



- **Separate Thread Used For**
 - Game Timer / Countdown – Updates the remaining time without interrupting gameplay.
 - GUI Updates – Keeps the interface responsive while processing player guesses.
 - Background Database Operations – Saves scores and streaks without delaying the game screen.

- **Why is Multithreading Needed?**

Without multithreading:

- The Swing GUI may freeze while processing guesses or saving data.
- The countdown timer may stop updating.
- The game becomes less responsive, resulting in a poor user experience.

EXCEPTION HANDLING PLAN

Failure Scenario	Exception Type	Handling Strategy
Player enters an invalid word (not 5 letters or not in dictionary)	<code>IllegalArgumentException</code>	Display an error message and ask the player to enter a valid word.
Database connection fails while saving scores	<code>SQLException</code>	Show a database error message and continue the game without saving.
Dictionary file is missing or cannot be loaded	<code>FileNotFoundException</code> / <code>IOException</code>	Display an error message and prevent the game from starting until the file is available.
Player submits an empty or null input	<code>NullPointerException</code>	Validate the input and prompt the player to enter a word.

Exception Handling Flow

User Action → Validate Input → Exception Occurs? → Handle with try-catch → Display Error Message → Continue Game



WORK PLAN / TIMELINE TO REVIEW 2



Week	Milestone
Week 1	Design the Java Swing GUI and implement the Word Dictionary Manager.
Week 2	Develop the Guess Validation Engine with color-coded tile feedback.
Week 3	Integrate JDBC with MySQL to store player scores and winning streaks.
Week 4	Implement multithreading, exception handling, testing, and bug fixing.
Review 2	Present the fully functional Wordle Clone with all modules integrated and tested.



ANTICIPATED CHALLENGES



Challenge

Planned Approach

Accurate Guess Validation

Implement a character-by-character comparison algorithm to correctly identify Green, Yellow, and Gray feedback.

Random Word Selection

Use **ArrayList**, **HashSet**, and the **Random** class to ensure valid words are selected without immediate repetition.

Maintaining GUI Responsiveness

Use **multithreading (Swing Timer/Thread)** to keep the interface responsive while handling timers and database operations in the background.

Expected Outcome

- Accurate and reliable word validation with correct color-coded feedback.
- A responsive and user-friendly interface, even during database operations.
- A stable, well-tested application that demonstrates OOP, Collections, Swing GUI, Exception Handling, Multithreading, and JDBC effectively.

Thank You!

- Any Questions?